

VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Chosen Software: Git

Student Name:	Mausam Kar
Registration Number:	24BAI10284
Course:	Open Source Software
Unit Coverage:	1 – 5
Date of Submission:	March 19, 2026

Course: Open Source Software | Max Marks: 100

Contents

1	Introduction & Overview	2
1.1	Chosen Software: Git	2
2	Research Analysis	3
2.1	Part A — Origin and Philosophy	3
2.1.1	The Problem Before Git	3
2.1.2	The Origin Story of Git	3
2.1.3	License Analysis: GPL v2	3
2.1.4	The Ethics of Open Source	4
2.2	Part B — Linux Footprint	4
2.2.1	Installation Method	4
2.2.2	Directory Structure	4
2.2.3	User and Permissions	5
2.2.4	Service Behavior	5
2.2.5	Update Model	5
2.3	Part C — The FOSS Ecosystem	5
2.3.1	Dependencies	5
2.3.2	What Git Enables & The Web Ecosystem	5
2.3.3	Community and Governance	6
2.4	Part D — Open Source vs Proprietary	6
3	Practical Shell Scripts	7
3.1	Script 1: System Identity Report	7
3.2	Script 2: Package Inspector	8
3.3	Script 3: Disk and Permission Auditor	10
3.4	Script 4: Log File Analyzer	11
3.5	Script 5: Manifesto Generator	13
4	Conclusion	15

Chapter 1

Introduction & Overview

When we think about huge apps or websites today, we usually picture big tech companies building everything behind closed doors. But actually, a massive part of the internet runs on something completely different: Open Source software. Open source isn't just about getting free stuff; it's a totally different way of sharing knowledge. When developers around the world can look at, fix, and improve the same code, things get built faster and become way more secure than if they were hidden away.

For this project, I chose to look into **Git**. Even though I use Git all the time to save my code for classes, I never really stopped to think about how it actually works or why it was made. We just type `git commit` without knowing the history behind it. In this report, I will talk about where Git came from, why its open-source license matters, and share some shell scripts I wrote to see how things run on Linux.

1.1 Chosen Software: Git

Git is what we call a distributed version control system. Basically, it's a tool that lets developers track changes to their files over time, work together on group projects without deleting each other's work, and easily undo mistakes if something breaks. Unlike older tools, Git doesn't need to connect to one main central server to do its job. Everyone gets their own full, working copy of the project history right on their laptop.

Git is basically everywhere now. Websites like GitHub and GitLab host millions of open-source projects using it. But what makes it so cool isn't just that it's popular—it's how it was made. It wasn't built by a corporation to sell user subscriptions. It was coded really quickly just to fix a genuine problem the Linux team was having, and then it was shared with the entire world for free.

Chapter 2

Research Analysis

2.1 Part A — Origin and Philosophy

2.1.1 The Problem Before Git

Before Git existed, managing code versions was pretty painful. People used systems like CVS or SVN. These were centralized, meaning the code lived on one main server. If you wanted to save your work, you needed an internet connection to that server. If the server ever went down or crashed, nobody could work. Doing simple things like branching (splitting code to test a new feature) caused massive headaches.

The story of Git actually starts directly with Linux. Linus Torvalds and his team were building the Linux operating system using a closed-source tool called BitKeeper because the company let them use it for free. But in 2005, there was some drama. Someone in the community tried to reverse-engineer how BitKeeper worked, so the company got mad and revoked the free licenses. Suddenly, the Linux team had no way to manage their massive project.

2.1.2 The Origin Story of Git

Instead of paying for an expensive enterprise tool or going back to the old, clunky centralized systems, Linus decided to just write his own. In April 2005, he started coding Git. Because he knew exactly what the Linux team needed, he got the first version working in just a couple of weeks.

He wanted it to follow a few simple rules:

- **Speed:** It had to be incredibly fast because the Linux kernel has millions of lines of code.
- **Distributed:** No single point of failure; every developer needed their own full copy.
- **Data Safety:** It uses special cryptographic hashes so that once code is saved, it can't be secretly corrupted or changed.

2.1.3 License Analysis: GPL v2

Git uses the GNU General Public License version 2 (GPL v2), which is the exact same license used for Linux. The Free Software Foundation says truly "free" software needs four freedoms: the freedom to run the program, the freedom to study and change the code, the freedom to share it, and the freedom to share your improvements. Git checks all those boxes perfectly.

A lot of people think companies can't sell open-source software, but they can! The catch is that GPL is a "copyleft" license. If a company modifies Git and shares it, they have to share their new source code under the GPL too. You can't just take the community's hard work, add a feature, and hide the code to sell it.

This is very different from the MIT License, which is way more relaxed and lets people do whatever they want (even close the source code) as long as they don't sue the original creator. Because Git is so important to how we build software, the GPL is the perfect choice to stop giant companies from monopolizing it. If I were building a big tool today, I would definitely pick GPL to protect it.

When developers talk about "free as in beer versus free as in speech", they mean two different things. A free app on your phone is "free as in beer": it costs zero dollars, but it's locked down and you don't actually own it. Open-source software is "free as in speech": you have the liberty and rights to see how it works and change it.

2.1.4 The Ethics of Open Source

Ethically, I think foundational tools like operating systems, web servers, and databases should always be open source so everyone benefits. But I also think it's okay for specific apps, like video games, to be closed source so developers can actually make a living.

It is totally fine for huge companies to use open source to make money, as long as they give back by funding the projects or hiring the developers who maintain them. In programming, we say we "stand on the shoulders of giants." This just means we aren't starting from scratch. When I write a script, it runs on Linux, using tools someone else built years ago. We literally build everything on top of the free gifts left by past developers.

2.2 Part B — Linux Footprint

This section briefly looks at how Git lives and behaves on a standard Linux computer.

2.2.1 Installation Method

Installing Git on a Linux machine like Ubuntu is super easy. You just use the default APT package manager:

```
1 sudo apt update
2 sudo apt install git
```

2.2.2 Directory Structure

The main executable program sits cleanly in `/usr/bin/git`. Whenever you use it in a project folder, it creates a hidden `.git` folder right there to store all the actual tracking history,

keeping everything isolated from the rest of your system. Your personal settings, like your email and name, are kept quietly in a `~/.gitconfig` file inside your home folder.

2.2.3 User and Permissions

Git operations run purely under the permissions of whoever is logged in. It never demands root (admin) privileges to do daily commits or formatting. This matters a lot for security because if a normal user accidentally runs a destructive Git command, they can only ruin their own files, not the whole server.

2.2.4 Service Behavior

Instead of running as a heavy background service constantly eating up RAM, Git is just a Command Line Interface (CLI) tool. It only wakes up, processes your command, and immediately goes back to sleep when you type something.

2.2.5 Update Model

Security patches and bug fixes are continuously maintained by the open-source community. These updates are then safely bundled and handled securely through the OS package manager (like `apt upgrade`), so you never have to hunt down sketchy installers from the internet.

2.3 Part C — The FOSS Ecosystem

2.3.1 Dependencies

Since Git was built in C, it depends heavily on standard GNU system tools. It also relies on external libraries like `OpenSSL` to handle secure internet connections and `zlib` for zipping files efficiently.

2.3.2 What Git Enables & The Web Ecosystem

Git completely changed the software industry. It directly enabled platforms like GitHub and GitLab. More importantly, it forms the backbone of the modern web ecosystem (like LAMP stacks). Without Git tracking exactly what changed in the code, automated tools wouldn't know when to safely test or deploy new updates to a live website. It acts as the invisible glue holding DevOps together.

2.3.3 Community and Governance

Despite running massive platforms like GitHub, Git's own internal development community is incredibly old-school. Updates and patches are still discussed by developers passing plain-text emails back and forth on a giant mailing list. It functions as an open, transparent dictatorship overseen by a core maintainer.

2.4 Part D — Open Source vs Proprietary

We can compare Git with an older proprietary tool like Microsoft's Team Foundation Version Control (TFVC).

Feature	Git (Open Source GPL)	TFVC (Proprietary)
Cost	100% Free. You can use it as much as you want without user limits.	Starts free but has very expensive paid tiers for bigger enterprise setups.
Security	Code is double-checked by thousands of smart people online.	Only Microsoft employees can look at the code to fix internal bugs.
Support	You search Google or Reddit, and read community forums.	You get official corporate IT support from Microsoft.
Flexibility	You can do everything entirely offline on a laptop on an airplane.	You need to be connected to the central server to do major work.
Control	You fully control your own local codebase history.	The central server controls the master history timeline.

Overall, I would absolutely recommend Git for any real-world scenario. It's free, it works fully offline, and essentially everyone uses it. Even Microsoft eventually gave up pushing TFVC and switched their own Windows development teams over to Git. While I probably won't be writing raw C code to contribute to Git's core anytime soon, I am happy to support open source by using it properly and sharing my own projects freely!

Chapter 3

Practical Shell Scripts

For this project, I wrote five shell scripts to practice using the Linux terminal. It helped me learn how to use loops, if-statements, and command line tools to extract data cleanly.

3.1 Script 1: System Identity Report

This is a simple script that grabs basic info about the computer. It uses variables and the \$(command) syntax to run things like uname and uptime, and then prints them out to introduce the system.

```
1  #!/bin/bash
2  # Script 1: System Identity Report
3  # Author: Mausam Kar | Course: Open Source Software
4
5  # --- Variables ---
6  STUDENT_NAME="Mausam Kar"
7  SOFTWARE_CHOICE="Git"
8
9  # --- System info ---
10 KERNEL=$(uname -r)
11 USER_NAME=$(whoami)
12 UPTIME=$(uptime -p)
13 DISTRO=$(uname -o)
14 HOME_DIR=$HOME
15 CURRENT_DATE=$(date)
16
17 # --- Display ---
18 echo "===== "
19 echo " Open Source Audit - $STUDENT_NAME "
20 echo "===== "
21 echo "Software Choice : $SOFTWARE_CHOICE"
22 echo "Distribution      : $DISTRO"
23 echo "Kernel            : $KERNEL"
24 echo "Current User      : $USER_NAME"
25 echo "Home Directory    : $HOME_DIR"
26 echo "Uptime            : $UPTIME"
27 echo "Date & Time       : $CURRENT_DATE"
28 echo "----- "
29 echo "License Message:"
30 echo "This Linux operating system is distributed under the GPL license."
```



```

31 echo "===== "
32
33 # Prevent terminal close on Windows
34 echo " "
35 read -p "Press [Enter] to exit..."

```

Listing 3.1: 01_system_identity.sh

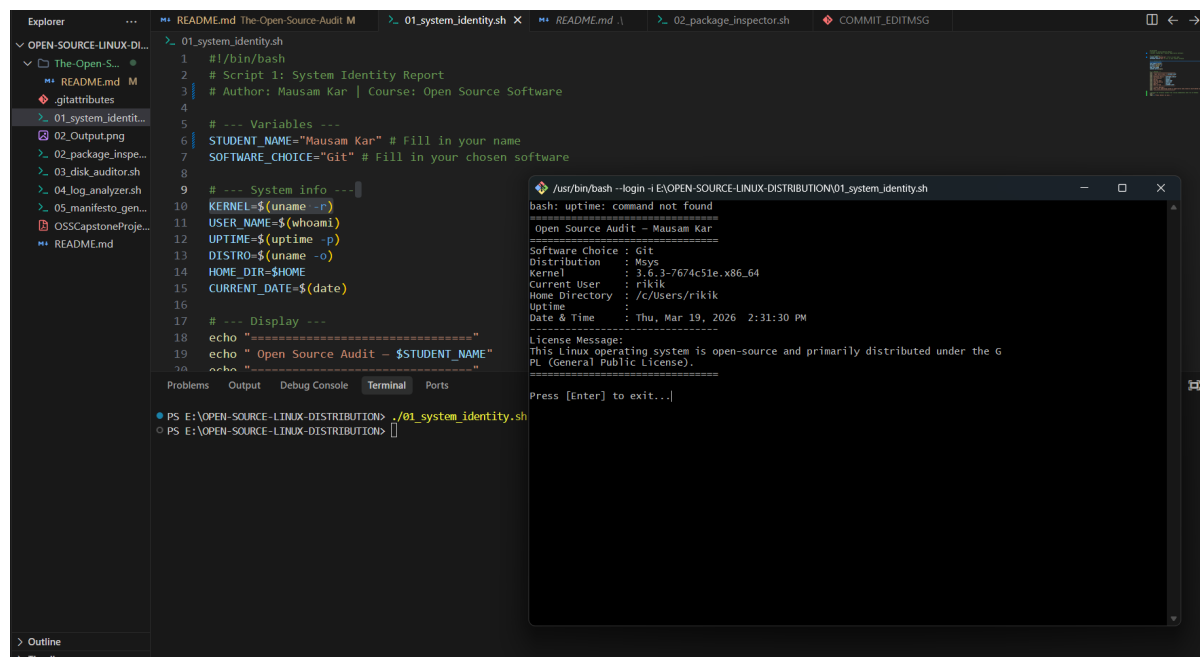


Figure 3.1: Output showing system info and the GPL license message.

3.2 Script 2: Package Inspector

This script checks if a piece of software is installed. It uses an 'if-else' block to look for the application package, and then uses a 'case' statement to print out a fun fact depending on what the software is.

```

1  #!/bin/bash
2  # Script 2: FOSS Package Inspector
3
4  PACKAGE="git"
5
6  if dpkg -l "$PACKAGE" &>/dev/null || command -v "$PACKAGE" &>/dev/null;
7  then
8      echo "$PACKAGE is installed."
9      if command -v dpkg &>/dev/null && dpkg -s "$PACKAGE" &>/dev/null; then
10         dpkg -s "$PACKAGE" | grep -E '^Version|^Description' | head -n 2
11     else

```

```

1  echo "Version: $($PACKAGE --version)"
2  fi
3  else
4  echo "$PACKAGE is NOT installed."
5  fi
6
7  case $PACKAGE in
8    httpd|apache2) echo "Apache: the web server that built the open
        internet" ;;
9    mysql) echo "MySQL: open source at the heart of millions of apps" ;;
10   git) echo "Git: decentralized tool that enabled global code
        collaboration" ;;
11   vlc) echo "VLC: media player that proves open source can handle any
        format" ;;
12   firefox) echo "Firefox: browser fighting for a free, open, and private
        web" ;;
13   *) echo "$PACKAGE: Another great tool in the open source ecosystem" ;;
14  esac
15
16  echo ""
17  read -p "Press [Enter] to exit..."

```

Listing 3.2: 02_package_inspector.sh

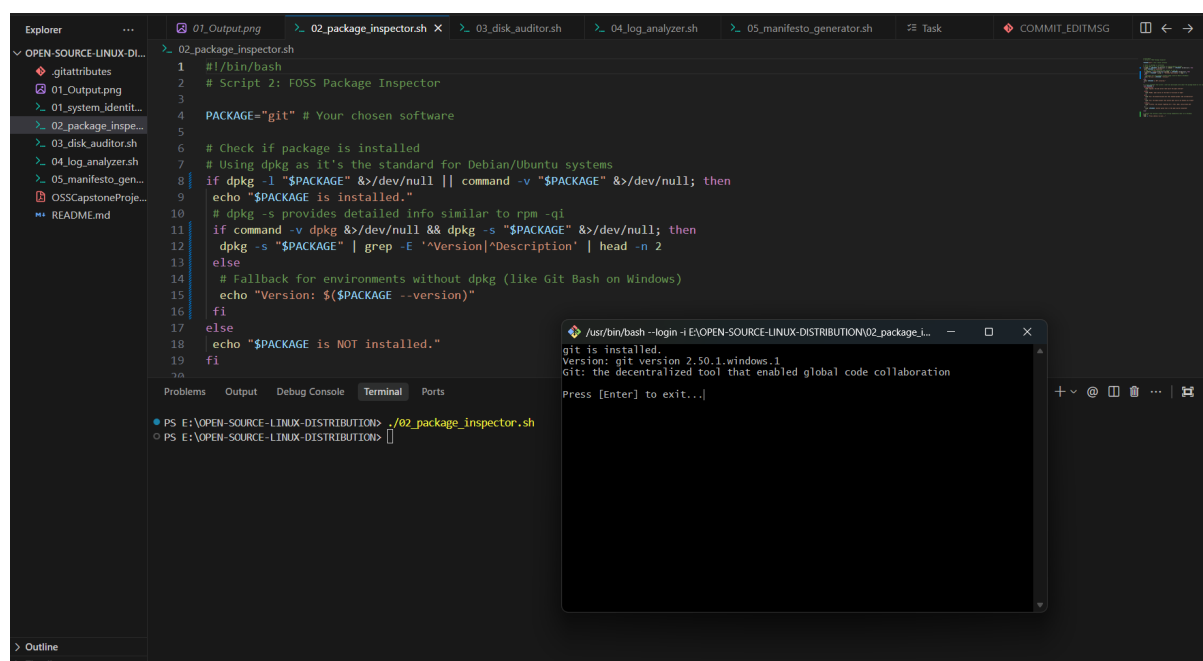


Figure 3.2: Script checking for Git and outputting its custom message.

3.3 Script 3: Disk and Permission Auditor

This script uses a `for` loop to check a list of folders. It runs commands like `ls -ld` and `df -h`, and then uses `awk` and `cut` to slice up the text so we only see the exact file sizes and permissions we actually care about.

```
1  #!/bin/bash
2  # Script 3: Disk and Permission Auditor
3
4  DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
5  echo "Directory Audit Report"
6
7  for DIR in "${DIRS[@]}; do
8      if [ -d "$DIR" ]; then
9          # Extract Permissions
10         PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
11         # Check directory size using du and cut
12         SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
13         # Check filesystem storage usage using df and awk
14         FS_USAGE=$(df -h "$DIR" 2>/dev/null | awk 'NR==2 {print $5}')
15
16         echo "$DIR => Permissions: $PERMS | Size: $SIZE | Disk Used: $FS_USAGE"
17
18     else
19         echo "$DIR does not exist on this system"
20     fi
21 done
22
23 GIT_CONFIG="$HOME/.gitconfig"
24 if [ -f "$GIT_CONFIG" ]; then
25     GIT_PERMS=$(ls -ld $GIT_CONFIG | awk '{print $1, $3, $4}')
26     echo "Git config ($GIT_CONFIG) exists. Permissions: $GIT_PERMS"
27 else
28     echo "Git config ($GIT_CONFIG) does not exist."
29 fi
30
31 echo ""
32 read -p "Press [Enter] to exit..."
```

Listing 3.3: 03_disk_auditor.sh

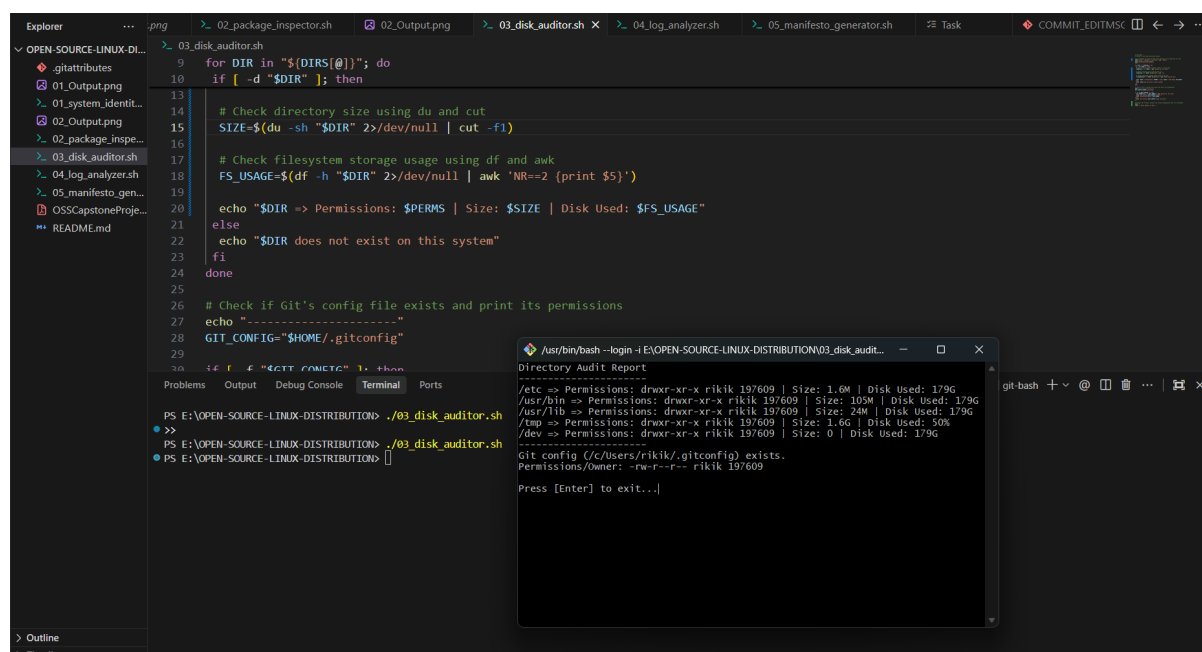


Figure 3.3: The loop returning the chopped up permission and filesize text.

3.4 Script 4: Log File Analyzer

This script takes arguments from the command line (\$1 and \$2). It uses a `while read` loop to scan through a huge file line by line to count how many times a word like "error" shows up, which is great for finding bugs.

```

1  #!/bin/bash
2  # Script 4: Log File Analyzer
3
4  LOGFILE=$1
5  KEYWORD=${2:-"error"}
6  COUNT=0
7
8  if [ ! -f "$LOGFILE" ]; then
9      echo "Error: File $LOGFILE not found."
10     read -p "Press [Enter] to exit..."
11     exit 1
12 fi
13
14 RETRY=0
15 while true; do
16     if [ -s "$LOGFILE" ]; then break; fi
17     echo "Warning: File is currently empty. Retrying in 2 seconds..."
18     sleep 2
19     RETRY=$((RETRY + 1))

```

```

20
21     if [ $RETRY -ge 3 ]; then
22         echo "Error: File is still empty. Exiting."
23         read -p "Press [Enter] to exit..."
24         exit 1
25     fi
26 done
27
28 while IFS= read -r LINE; do
29     if echo "$LINE" | grep -iq "$KEYWORD"; then COUNT=$((COUNT + 1)); fi
30 done < "$LOGFILE"
31
32 echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
33
34 if [ $COUNT -gt 0 ]; then
35     echo "--- Last 5 Matching Lines ---"
36     grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
37 fi
38
39 echo ""
40 read -p "Press [Enter] to exit..."

```

Listing 3.4: 04_log_analyzer.sh

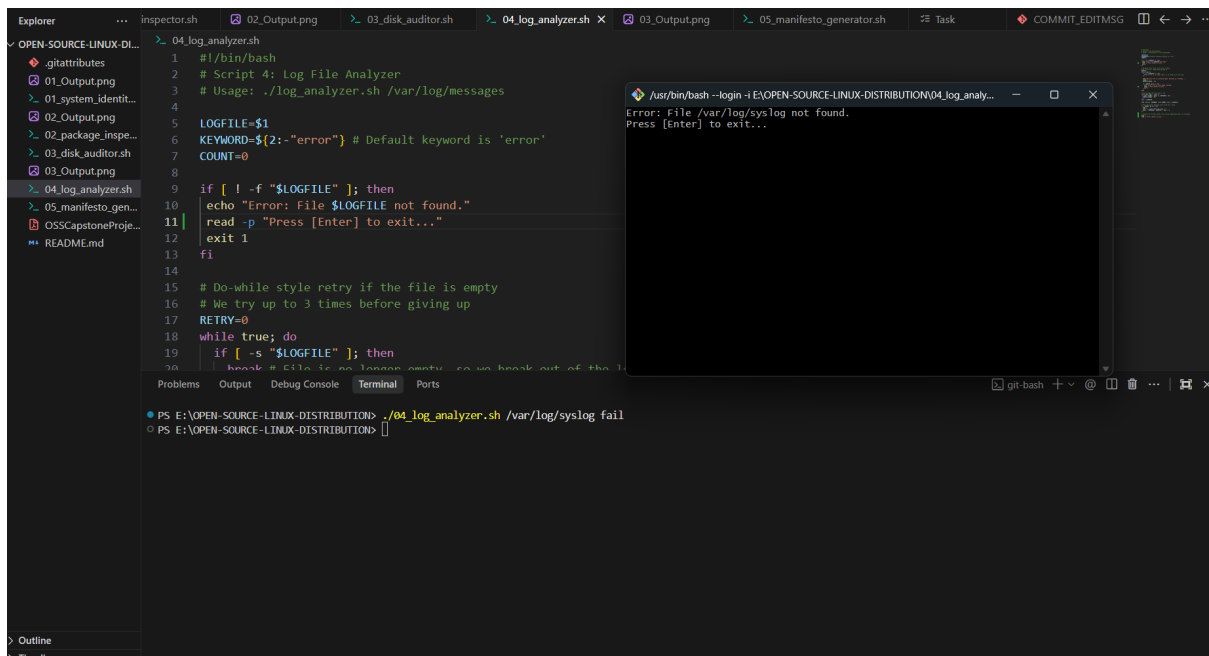


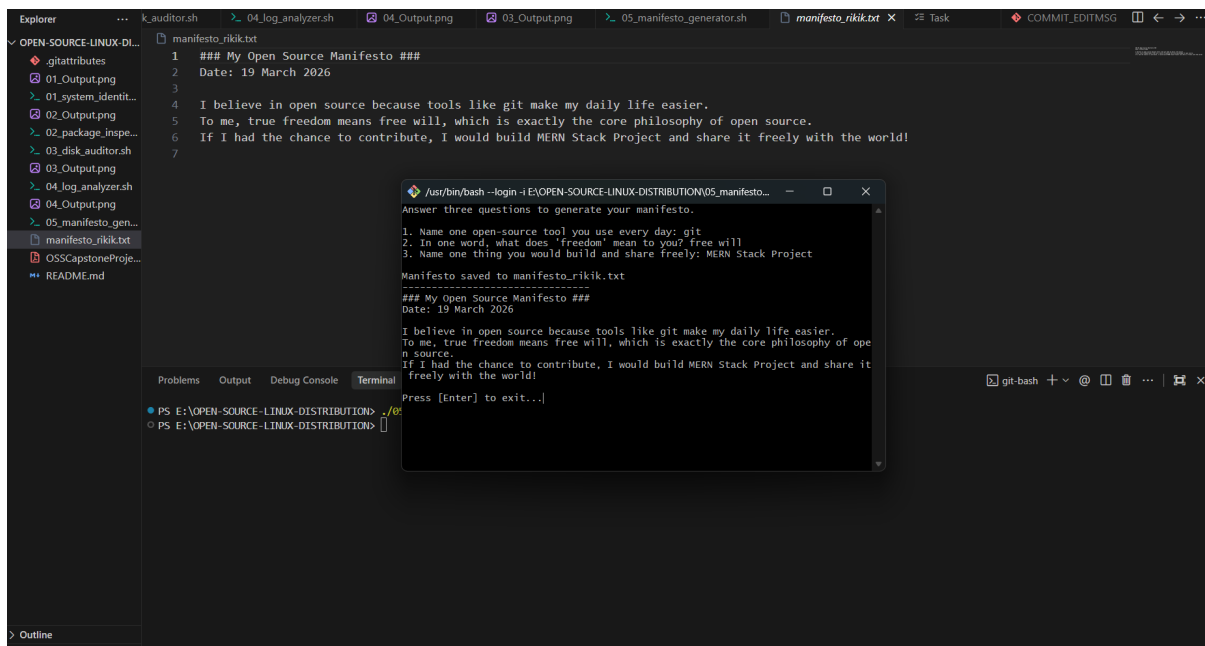
Figure 3.4: The script catching an error condition and printing the summary.

3.5 Script 5: Manifesto Generator

This is an interactive script that uses `read` to ask the user questions while it runs in the terminal. Once it gets your answers, it glues them together into a paragraph and uses `>>` redirection to save it all into a new `.txt` file.

```
1  #!/bin/bash
2  # Script 5: Open Source Manifesto Generator
3
4  echo "Answer three questions to generate your manifesto."
5  read -p "1. Name one open-source tool you use every day: " TOOL
6  read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
7  read -p "3. Name one thing you would build and share freely: " BUILD
8
9  DATE=$(date '+%d %B %Y')
10 OUTPUT="manifesto_$(whoami).txt"
11
12 echo "### My Open Source Manifesto ###" > "$OUTPUT"
13 echo "Date: $DATE" >> "$OUTPUT"
14 echo "I believe in open source because tools like $TOOL make my daily
    life easier." >> "$OUTPUT"
15 echo "To me, true freedom means $FREEDOM, which is exactly the core
    philosophy of open source." >> "$OUTPUT"
16 echo "If I had the chance to contribute, I would build $BUILD and share
    it!" >> "$OUTPUT"
17
18 echo "Manifesto saved to $OUTPUT"
19 cat "$OUTPUT"
20
21 echo ""
22 read -p "Press [Enter] to exit..."
```

Listing 3.5: 05_manifesto_generator.sh



The screenshot shows a VS Code editor with a file explorer on the left containing files like `01_Output.png`, `02_Output.png`, `03_Output.png`, `04_Output.png`, `05_manifesto_generator.sh`, `manifesto_rikik.txt`, and `README.md`. The main editor displays the content of `manifesto_rikik.txt`, which includes a header, a date, and a paragraph about open source. A terminal window is open in the foreground, showing the execution of a script that prompts for three questions and then displays the generated manifesto text, which matches the content in the text file.

```
1 ### My Open Source Manifesto ###
2 Date: 19 March 2026
3
4 I believe in open source because tools like git make my daily life easier.
5 To me, true freedom means free will, which is exactly the core philosophy of open source.
6 If I had the chance to contribute, I would build MERN Stack Project and share it freely with the world!
7
```

```
Answer three questions to generate your manifesto.
1. Name one open-source tool you use every day: git
2. In one word, what does 'freedom' mean to you? free will
3. Name one thing you would build and share freely: MERN Stack Project

Manifesto saved to manifesto_rikik.txt
-----
### My Open Source Manifesto ###
Date: 19 March 2026

I believe in open source because tools like git make my daily life easier.
To me, true freedom means free will, which is exactly the core philosophy of open source.
If I had the chance to contribute, I would build MERN Stack Project and share it freely with the world!

Press [Enter] to exit...
```

Figure 3.5: The script printing the newly generated text file onto the screen.

Chapter 4

Conclusion

This capstone project was really eye-opening for me. Before this, the Linux terminal and bash scripting felt pretty intimidating. But actually building these scripts to check folders and read log files showed me how simple utilities like `awk` and `grep` can be combined to do really powerful things.

Learning about the GPL license and why Linus Torvalds built Git also made me appreciate open source a lot more. It's not just a nerdy hobby; it's the actual engine that makes modern internet development possible. It feels pretty cool knowing that whatever I build in the future will be running on the same free tools that the whole community shares and maintains.